VARUNA — Approach Document

SIH26067 · Web-based Interactive 3D Visualization Platform for Ocean Model & In-Situ Data

Ministry of Earth Sciences (MoES) · Software Track · Theme: Smart Automation

VARUNA = *Visualization & Assimilation of Real-time Underwater Network Analytics*

1. The Problem, Explained Properly

There are two separate "versions" of the ocean that currently don't talk to each other visually:

Version A — The simulated ocean. INCOIS runs the Indian Ocean Forecast System (INDOFOS), built on the ROMS/HYCOM ocean models. It computes a dense 3D+time grid (lat × lon × depth × time) of currents, waves, sea surface temperature, and mixed layer depth, forecasting 5–10 days ahead at 3-hour intervals. This is a *simulation* — physics equations solved on a grid, not a direct measurement.

Version B — The real ocean. Argo floats, moored buoys (NIOT), XBT casts, drifters, and ADCP moorings physically measure temperature, salinity, and currents at scattered points. Sparse in space, irregular in time, but *ground truth*.

The actual gap: validating the simulation against reality today is a manual, periodic process — scientists compare snapshots in desktop tools (MATLAB, ParaView, Panoply). There is no single interactive, browser-based, 3D space where both versions of the ocean sit together, continuously, for anyone — researcher, naval officer, or fisherman — to explore and trust.

Who needs this: fishermen, Indian Navy, Coast Guard, shipping/offshore operators, coastal disaster management authorities, and ocean researchers validating models.

2. Our One-Line Pitch

A browser-based **living digital twin** of the Indian Ocean that doesn't just display INDOFOS forecasts next to Argo/buoy readings — it **intelligently fuses** them and **actively flags** when the model and reality disagree.

This is the difference between "a viewer" (what most existing tools are) and "a validation and early-warning system" (what nothing currently is, in a single accessible tool).

3. System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│ DATA SOURCES                                                 │
│ ROMS / HYCOM reanalysis (NetCDF, gridded model output)       │
│ Argo floats · moored buoys · XBT · drifters (real point data)│
└──────────────────────────────┬───────────────────────────────┘
         ▼
┌──────────────────────────────────────────────────────────────┐
│ INGESTION & PROCESSING                                       │
│ xarray / netCDF4 parsers → regrid to common resolution       │
│ → chunk into lightweight JSON / GeoJSON tiles (FastAPI)      │
└──────────────────────────────┬───────────────────────────────┘
         ▼
┌──────────────────────────────────────────────────────────────┐
│ INTELLIGENCE LAYER ← the differentiator                      │
│ ① Graph Fusion Engine — corrects the model grid using a      │
│   graph of real sensor readings                              │
│ ② Drift Memory Engine — running per-region "surprise" score, │
│   flags model-vs-reality divergence live                     │
│ ③ Nowcast Engine (stretch) — short-range latent prediction   │
└──────────────────────────────┬───────────────────────────────┘
         ▼
┌──────────────────────────────────────────────────────────────┐
│ API LAYER — REST + WebSocket                                 │
│ /model /observations /fused /alerts (live push)              │
└──────────────────────────────┬───────────────────────────────┘
         ▼
┌──────────────────────────────────────────────────────────────┐
│ 3D VISUALIZATION — CesiumJS / Three.js                       │
│ Draped ocean surfaces · animated current vectors ·           │
│ time-scrubber · click-to-inspect · live alert feed panel     │
└──────────────────────────────────────────────────────────────┘
```

4. Research Landscape → What We're Actually Using

The four techniques that made the cut — each one earns its place in the architecture, nothing here is a buzzword add-on.

Research area	What it actually does	How it's used here
GNN (Graph Neural Networks)	Graph-structured reasoning over relational data — nodes and edges instead of a fixed grid	Powers the **Graph Fusion Engine**: in-situ sensors become graph nodes, edges weighted by real oceanographic connectivity (not straight-line distance), correcting the dense model grid. This is the core differentiator.
Titans	Neural long-term memory that updates itself at inference time via a "surprise" signal	Simplified into our **Drift Memory Engine**: a rolling divergence ("surprise") score per ocean region between model output and latest observation, firing live alerts when it crosses a threshold. The other core differentiator.
Mamba / SSM (State Space Models)	Linear-time sequence recurrence, replaces attention, constant memory during generation	Backs our real-time buoy/Argo ingestion design: new sensor pings update state in linear time, no quadratic blow-up as the sensor network grows
JEPA / V-JEPA2	Predicts future states in a compressed *latent* space instead of raw pixels/grid values	Powers the optional **Nowcast Engine** (stretch goal): a lightweight encoder-decoder predicting a compressed next-24h ocean state, framed honestly as "inspired by," not the full research pipeline

5. Tech Stack

Layer	Choice	Why
3D Visualization	**CesiumJS** (globe/geospatial) or **Three.js** (lighter, region-scoped)	Standard for browser-based geospatial 3D; used in peer-reviewed ocean-visualization frameworks for this exact problem type
Backend API	**FastAPI**	Fast to stand up, async-friendly for WebSocket alert push
Data processing	**xarray + netCDF4**	Standard scientific Python stack for gridded NetCDF ocean data
Graph fusion	**PyTorch Geometric** (fallback: manual graph-weighted correction with `networkx`)	PyG handles GNN math directly; fallback keeps us shippable if training proves unstable in the timeline

Layer	Choice	Why
Real-time alerts	**WebSockets**	Live push of drift/anomaly alerts to the UI without polling
Demo data sources	**Copernicus Marine (GLORYS reanalysis)**, **NOAA HYCOM public THREDDS**, **Argo via Argovis API / Coriolis GDAC**	Real, publicly accessible equivalents to INDOFOS-class data — live INCOIS operational feeds require authorization we won't have in this window

Framing for judges: "*Designed to ingest INCOIS/INDOFOS operational feeds; demo runs on equivalent open reanalysis data given the build window.*" Honest framing lands better than vague claims of live government data access.

⚠ **UPDATE — Dev environment note (added after team check-in): the official `mamba-ssm` PyTorch package is NOT supported on native Windows** — it needs custom CUDA kernels that fail to build on Windows 11 (confirmed open issue on the state-spaces/mamba GitHub repo) and requires an NVIDIA GPU regardless of OS. Workarounds exist (WSL2, or unofficial community-built wheels) but aren't worth the risk mid-sprint. **Decision: we do NOT install the mamba-ssm library at all.** Our use of Mamba/SSM is conceptual, not literal — we only need a small custom linear-time recurrent state-update function (~20 lines of plain PyTorch/NumPy) for the ingestion layer, which runs identically on Windows/Mac/Linux, CPU-only, no compiled extensions. Same architectural story on the slide, zero install risk. PyTorch Geometric (Graph Fusion Engine) is unaffected — it installs and runs fine CPU-only on Windows.

Scope lock for the demo: Bay of Bengal region, three variables — Sea Surface Temperature, Currents (u/v), Wave Height. Matches INDOFOS's own core outputs and keeps data volume manageable.

6. Key Differentiator Features (what judges actually remember)

Trust overlay — every point on the ocean surface color-coded by confidence: green where model matches real sensors, red where they diverge. Makes the fusion engine *visible*, not a backend claim.
Live alert feed — scrolling panel: "*Drift detected: Bay of Bengal, SST diverging 1.8°C from forecast — possible marine heatwave.*" The Drift Memory Engine surfaced as a product feature.
Expert / Public mode toggle — flip between a dense scientific panel (raw values, depth profiles, confidence stats) and a simplified public view ("safe" / "caution" / "avoid"). Answers "who is this for" in one click.
Time-scrubber — drag through the forecast window, watch predicted vs. actual converge or diverge live on stage.

7. Delivery Timeline

Stage 1 — Internal SIH (i-SIH), Round 1: Pitch (now)

PPT + verbal explanation only. Nothing needs to be built yet — this document + the 6-slide official format is the deliverable.

Stage 2 — Internal SIH, Round 2: Offline, 70% working (2-week sprint)

This is the real build pressure. Detailed day-by-day plan below.

Stage 3 — Push to 100% (post-selection, before national portal submission ~30 Sept)

Polish, add the Nowcast Engine if time allows, record a clean backup demo video, finalize the idea PPT.

Stage 4 — National screening (October)

Idea evaluated on originality, scalability, technical implementation, real-world impact.

Stage 5 — Mentoring window (if shortlisted, Oct–Dec)

Refine with expert/faculty mentors, harden the prototype, expand robustness.

Stage 6 — Grand Finale (December, 36-hour non-stop build)

Integration, polish, live-data hookup, pitch rehearsal — not building from zero.

8. The 2-Week Sprint Plan (Round 2 target: 70% working)

Three people, three parallel tracks from day one — nobody blocks anybody, everyone builds against agreed mock data/API contracts first.

Days	Person 1 — Visualization	Person 2 — Data & Backend	Person 3 — Intelligence
1–2	Scene setup, camera controls, UI shell	Repo/API contract setup, pick region + 3 variables	Design graph structure for sensor network
3–6	Render static ocean surface with mock data	Ingest real NetCDF (Copernicus/HYCOM) for chosen region; stand up `/model` endpoint	Build graph-weighted correction against mock model grid; stub `/fused` endpoint
7–8	**Checkpoint:** wire viz to P2's real `/model` endpoint	Pull real Argo/buoy data for same region/window	—
9–10	**Checkpoint:** wire `/fused` endpoint in as toggleable trust overlay	Support fused-data serving	Finalize graph fusion correction
11–12	Build live alert feed panel UI	Add `/alerts` endpoint + WebSocket push	Build rolling divergence ("surprise") score
13	Polish pass — time-scrubber, expert/public toggle	Bug fixes, stability pass	Bug fixes, threshold tuning
14	Full rehearsal, record backup demo video, prep pitch script (shared across all three)		

What "70%" honestly means here: working 3D viewer + real model data + real in-situ data + one working intelligence engine (graph fusion) + basic drift alerts. The Nowcast Engine stays a documented future-scope slide unless there's slack time.

9. Work Distribution — Ownership

Person 1 — Visualization Owner: CesiumJS/Three.js scene, time-scrubber, layer toggles, trust-overlay rendering, expert/public mode UI. Owns the "wow factor" on stage.
Person 2 — Data & Backend Owner: NetCDF ingestion, regridding, FastAPI endpoints, real Argo/buoy pipeline, WebSocket alert push. Owns data integrity and honesty of the demo.
Person 3 — Intelligence Owner: Graph fusion engine, drift/surprise scoring engine, nowcast stretch if time allows. Owns the technical differentiator judges will interrogate hardest.

All three must be able to explain, modify, and defend *every* part of the system — not just their own layer — since judges test this directly.

10. Risk & Fallback Plan (graceful degradation order)

If time runs short, cut in this order — never cut something that breaks the live demo:

Nowcast Engine (JEPA-inspired) — cut first, becomes a "Future Scope" slide bullet.
Full PyTorch Geometric GNN — fall back to the simpler graph-weighted correction; still legitimately "graph-based fusion" under questioning.
Expert/Public mode toggle — fall back to expert view only.
Multi-variable support — fall back to SST only if currents/waves aren't stable in time.
Never cut: the core loop (real model data + real in-situ data + at least one visible fusion signal). This is the non-negotiable floor — everything else is layered on top of it.

11. Business Case: Competitive Landscape & Current Trends

This isn't a hackathon buzzword — it's where global and Indian ocean policy is actively heading right now, in 2026. This section is your strongest ammunition for the Impact & Benefits and Feasibility slides.

Why now — the global trend

Digital Twin of the Ocean (DTO) is an active, funded international movement, not a hackathon concept. The EU's **EDITO** platform (Horizon Europe flagship, 2022–2028) and the global **DITTO** initiative under the **UN Decade of Ocean Science for Sustainable Development (2021–2030)** are building exactly this category of tool — the next DITTO Summit runs November 2026 in Yokohama.
A January 2026 National Science Review paper frames Digital Twins of the Ocean as the key unlock for the **$2.5 trillion global ocean economy**, calling for exactly the kind of AI + physical-model integration our Graph Fusion and Drift Memory engines represent.

Why now — India specifically (this is the strongest card)

MoES itself convened a panel titled *"AI for Our Oceans of Tomorrow: Data, Models and Governance"* at the India AI Impact Summit 2026 (Feb 2026, New Delhi). IMD's Director General publicly called for integrating AI-enabled models with traditional physical ocean models — that is, almost word-for-word, our core pitch.
Experts at that same summit stated India is positioned to **lead the Global South in a "Digital Ocean Infrastructure."**
Real government money is already moving here: the **Deep Ocean Mission** (~₹4,077 crore over 5 years) and India's **Blue Economy**, described as a $1 trillion+ opportunity.
INCOIS already runs an in-house **AI/ML-based El Niño prediction system** (reported to Parliament, August 2026) — proof the organization is actively receptive to AI-augmented forecasting, not resistant to it.

Competitive landscape — where VARUNA actually sits

Tool	3D?	Live model↔observation fusion?	India/INCOIS-specific?	Dual expert/public mode?
INCOIS OSF (current, deployed)	No — 2D GIS overlays	No	Yes	No
EU EDITO / Digital Twin Ocean	Partial — data/model access platform	Emerging	No — Europe-focused	No — expert-oriented
Academic research tools (pyParaOcean, GODIVA, Cesium-based frameworks)	Yes	No	No	No — research prototypes, never deployed
VARUNA (ours)	**Yes**	**Yes**	**Yes**	**Yes**

Closing line for the slide: *VARUNA isn't chasing a trend — it's a working entry point into a direction MoES has already publicly committed to. We're just the first to make it real, in 3D, for India specifically.*

12. References (for the Research & References slide)

INCOIS — Indian Ocean Forecast System (INDOFOS), ROMS/HYCOM operational setup

Copernicus Marine Service — GLORYS reanalysis product

NOAA HYCOM public THREDDS data server

Argovis API / Coriolis GDAC — Argo float data access

Titans: Learning to Memorize at Test Time (test-time neural memory)

V-JEPA2 — Joint Embedding Predictive Architecture for world modeling

Prior academic ocean-visualization frameworks: pyParaOcean, i4Ocean, GODIVA, Cesium/Plotly.js browser-based 3D ocean visualization frameworks

EU Digital Twin of the Ocean (EDITO), Horizon Europe — Mercator Ocean International

DITTO (Digital Twins of the Ocean), UN Decade of Ocean Science for Sustainable Development

MoES panel, "AI for Our Oceans of Tomorrow: Data, Models and Governance," India AI Impact Summit 2026

Government of India — Deep Ocean Mission, Ministry of Earth Sciences

Next: convert Sections 1–6 into the official 6-slide SIH idea format (Title → Proposed Solution → Technical Approach → Feasibility & Viability → Impact & Benefits → Research & References).